

Lecture 12: Heapsort vs Quicksort vs Mergesort Tradeoffs

Topics

- Heapsort: sorting with a heap in $O(n \log n)$ time, $O(1)$ space
- Mergesort: divide-and-conquer with guaranteed $O(n \log n)$
- Quicksort: fast average case, but $O(n^2)$ worst case
- Timsort: Python's hybrid algorithm
- Empirical benchmarks: which is fastest in practice?

Goals

- Implement heapsort using Python's `heapq`
- Understand mergesort's divide-and-conquer strategy
- Understand quicksort's partitioning algorithm
- Benchmark algorithms on real Spotify data
- Choose the right sorting algorithm for each scenario

Roadmap

First half (≈ 45 min)

- Heapsort: in-place sorting with a heap
- Implementation using `heapq`
- Mergesort: divide-and-conquer recursive sorting
- Timsort: Python's adaptive hybrid
- In-class exercise 1 (commit required)

Break (3 min)

Second half (≈ 45 min)

- Quicksort: partitioning and recursion
- Pivot selection strategies
- Empirical benchmarks on track data
- Algorithm comparison: space, stability, best/worst cases
- When to use which algorithm
- In-class exercise 2 (commit required)
- Complexity summary and wrap-up

Setup: load Spotify data and timing utilities

```
In [ ]: import csv
import heapq
import time
import random
from collections import Counter

# Load tracks
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        tracks[row['track_id']] = row

# Convert to list for sorting
track_list = list(tracks.values())

print(f"Loaded {len(tracks)} tracks")

# Timing helper
def time_sort(sort_func, data, key=None):
    """Time a sorting function."""
    data_copy = data.copy() # Don't modify original
    start = time.time()
    if key:
        result = sort_func(data_copy, key=key)
    else:
```

```
        result = sort_func(data_copy)
    elapsed = time.time() - start
    return elapsed, result
```

Part 1: Heapsort

Key idea: Use a heap to extract elements in sorted order.

Algorithm:

1. Build a min-heap from all elements: $O(n)$
2. Extract the minimum n times: $O(n \log n)$
3. Total: $O(n \log n)$

Properties:

- **Time:** $O(n \log n)$ worst case (guaranteed)
- **Space:** $O(1)$ extra space (can be in-place)
- **Stable?** No (relative order not preserved)
- **Adaptive?** No (doesn't exploit existing order)

When to use: Need guaranteed $O(n \log n)$ time with minimal space.

Heapsort implementation

```
In [ ]: def heapsort(items, key=None):
        """Sort items using a heap."""
        # Build heap: O(n)
        if key:
            # Store (key_value, index, item) to handle ties
            heap = [(key(item), i, item) for i, item in enumerate(items)]
        else:
            heap = items.copy()

        heapq.heapify(heap)

        # Extract min n times: O(n log n)
        result = []
        while heap:
            if key:
                _, _, item = heapq.heappop(heap)
                result.append(item)
            else:
                result.append(heapq.heappop(heap))

        return result

# Test on numbers
numbers = [5, 2, 8, 1, 9, 3]
sorted_nums = heapsort(numbers)
print(f"Heapsort: {numbers} → {sorted_nums}")
```

Test on tracks

```
sample_tracks = track_list[:10]
```

```
sorted_tracks = heapsort(sample_tracks, key=lambda t: t['name'])
```

```
print("\nHeapsort by track name:")
```

```
for t in sorted_tracks[:5]:
```

```
    print(f"    {t['name']}")
```


Heapsort: why it works

Min-heap property: Parent $\leq$ children

Invariant: The root is always the minimum element.

Algorithm steps:

Original: [5, 2, 8, 1, 9, 3]

1. Build heap:	[1, 2, 3, 5, 9, 8]	(heapify in $O(n)$)
2. Pop min (1):	[2, 5, 3, 8, 9]	result = [1]
3. Pop min (2):	[3, 5, 8, 9]	result = [1, 2]
4. Pop min (3):	[5, 9, 8]	result = [1, 2, 3]
5. Pop min (5):	[8, 9]	result = [1, 2, 3, 5]
6. Pop min (8):	[9]	result = [1, 2, 3, 5, 8]
7. Pop min (9):	[]	result = [1, 2, 3, 5, 8, 9]

Complexity: Build heap $O(n)$ + n pops $\times O(\log n)$ = $O(n \log n)$

Mergesort: Divide and Conquer

Key idea: Divide array in half, sort each half, merge sorted halves.

Algorithm:

1. **Divide:** Split array into two halves
2. **Conquer:** Recursively sort each half
3. **Combine:** Merge two sorted halves

Properties:

- **Time:** $O(n \log n)$ always (best, average, worst)
- **Space:** $O(n)$ for merge buffer
- **Stable?** Yes (maintains relative order)
- **Adaptive?** No (doesn't exploit existing order)

When to use: Need guaranteed $O(n \log n)$ and stability, have extra space.

Mergesort implementation

```
In [ ]: def mergesort(items, key=None):
        """Sort items using mergesort."""
        if len(items) <= 1:
            return items

        # Divide
        mid = len(items) // 2
        left = mergesort(items[:mid], key=key)
        right = mergesort(items[mid:], key=key)

        # Conquer: merge sorted halves
        return merge(left, right, key=key)

def merge(left, right, key=None):
    """Merge two sorted lists."""
    result = []
    i = j = 0

    # Compare and merge
    while i < len(left) and j < len(right):
        left_val = key(left[i]) if key else left[i]
        right_val = key(right[j]) if key else right[j]

        if left_val <= right_val:
            result.append(left[i])
            i += 1
```

```
        else:
            result.append(right[j])
            j += 1

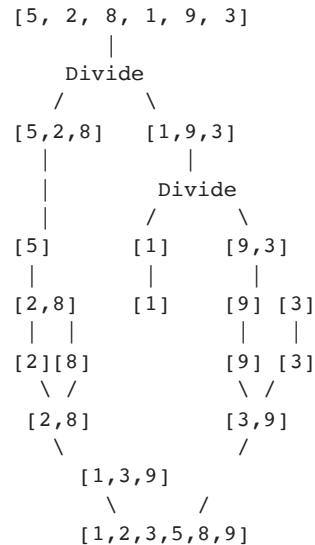
    # Append remaining
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test
numbers = [5, 2, 8, 1, 9, 3]
sorted_nums = mergesort(numbers)
print(f"Mergesort: {numbers} → {sorted_nums}")

# Test on tracks
sorted_tracks = mergesort(sample_tracks, key=lambda t: t['artist'])
print("\nMergesort by artist:")
for t in sorted_tracks[:5]:
    print(f"    {t['artist']}: {t['name']}")
```

Mergesort: why it works

Divide-and-conquer visualization:



Merging is $O(n)$: One pass through both lists.

Recursion depth is $O(\log n)$: Each level splits in half.

Total: $O(n \log n)$

Timsort: Python's hybrid

What is Timsort?

- Python's built-in sorting algorithm (used by `sorted()` and `.sort()`)
- Hybrid of **mergesort** and **insertion sort**
- Invented by Tim Peters in 2002

Key features:

1. **Adaptive:** Exploits existing order in data (runs)
2. **Stable:** Preserves relative order of equal elements
3. **Hybrid:** Uses insertion sort for small subarrays (<64 elements)
4. **Guaranteed $O(n \log n)$ worst case**
5. **$O(n)$ best case** on already-sorted or nearly-sorted data

Why it's fast in practice:

- Real-world data often has partial order
- Insertion sort is fast on small arrays
- Mergesort guarantees worst-case performance

Result: Best real-world performance of any general-purpose sort.

Exercise 1: Implement basic merge function

Task: Write a function to merge two sorted lists of tracks.

Requirements:

1. Function signature: `merge_tracks(left, right, key)`
2. Merge two sorted lists into one sorted list
3. Use the key function to compare items
4. Maintain stability (if `left[i] == right[j]`, take from left first)
5. Test with two sorted track lists (sorted by duration)

Hint: Use two pointers (i for left, j for right) and compare key values.

Commit required: Commit your solution before the break.

```
In [ ]: # YOUR CODE HERE
        # Implement merge_tracks function

def merge_tracks(left, right, key):
    """Merge two sorted lists of tracks."""
    # TODO: implement
    pass

# Test with two sorted lists
# left = sorted(track_list[:5], key=lambda t: int(t['duration_ms']))
```

```
# right = sorted(track_list[5:10], key=lambda t: int(t['duration_ms']))  
# merged = merge_tracks(left, right, key=lambda t: int(t['duration_ms']),
```


Break (3 minutes)

Commit your Exercise 1 solution now!

When we return:

- Quicksort and partitioning
- Empirical benchmarks on real data
- Algorithm comparison and selection guide

Part 2: Quicksort

Key idea: Pick a pivot, partition array around it, recursively sort partitions.

Algorithm:

1. **Pick pivot:** Choose an element (e.g., first, last, random, median)
2. **Partition:** Rearrange so elements $<$ pivot are left, $\geq$ pivot are right
3. **Recurse:** Sort left and right partitions

Properties:

- **Time:** $O(n \log n)$ average, $O(n^2)$ worst case
- **Space:** $O(\log n)$ for recursion stack
- **Stable?** No (standard version)
- **Adaptive?** Somewhat (good pivots help)

When to use: Need fast average-case performance, limited space.

Quicksort partitioning

```
In [ ]: def quicksort(items, key=None):  
        """Sort items using quicksort."""  
        if len(items) <= 1:  
            return items  
  
        # Pick pivot (middle element)  
        pivot = items[len(items) // 2]  
        pivot_key = key(pivot) if key else pivot  
  
        # Partition into three groups  
        left = []  
        middle = []  
        right = []  
  
        for item in items:  
            item_key = key(item) if key else item  
            if item_key < pivot_key:  
                left.append(item)  
            elif item_key > pivot_key:  
                right.append(item)  
            else:  
                middle.append(item)  
  
        # Recursively sort and concatenate  
        return quicksort(left, key=key) + middle + quicksort(right, key=key)
```

Test

```
numbers = [5, 2, 8, 1, 9, 3, 7, 4, 6]
```

```
sorted_nums = quicksort(numbers)
```

```
print(f"Quicksort: {numbers} → {sorted_nums}")
```

Test on tracks

```
sorted_tracks = quicksort(sample_tracks, key=lambda t: int(t['duration_r
```

```
print("\nQuicksort by duration:")
```

```
for t in sorted_tracks[:5]:
```

```
    duration_sec = int(t['duration_ms']) / 1000
```

```
    print(f"    {duration_sec:.1f}s: {t['name']}")
```

Quicksort: why it works

Partitioning visualization:

```
Original: [5, 2, 8, 1, 9, 3]
Pivot: 8 (middle element)

Partition:
  left:  [5, 2, 1, 3] (< 8)
  middle: [8]         (= 8)
  right: [9]          (> 8)

Recurse on left:
  [5, 2, 1, 3] → pivot=2
  left: [1], middle: [2], right: [5, 3]

Recurse on right: [5, 3] → pivot=3
  left: [], middle: [3], right: [5]

Result: [1, 2, 3, 5, 8, 9]
```

Average case: Pivot splits roughly in half → $O(n \log n)$

Worst case: Always pick min/max → $O(n^2)$ (already sorted!)

Pivot selection strategies

The pivot determines performance!

Strategy	Best Case	Worst Case	When to Use
First element	$O(n \log n)$	$O(n^2)$ on sorted data	Never (too risky)
Last element	$O(n \log n)$	$O(n^2)$ on sorted data	Never (too risky)
Middle element	$O(n \log n)$	$O(n^2)$ rare	Simple, usually good
Random element	$O(n \log n)$	$O(n^2)$ rare	Good for adversarial input
Median-of-three	$O(n \log n)$	$O(n^2)$ very rare	Best for typical data

Median-of-three: Pick median of first, middle, last elements.

Why it matters:

- Bad pivot (min/max) $\rightarrow$ unbalanced partitions $\rightarrow O(n^2)$
- Good pivot (median) $\rightarrow$ balanced partitions $\rightarrow O(n \log n)$
- Random pivot $\rightarrow O(n \log n)$ expected (probabilistic guarantee)

Empirical benchmarks on Spotify data

Let's measure real performance!

```
In [ ]: # Benchmark on different input sizes
        sizes = [100, 500, 1000, 2000]

        print("Benchmark: Sorting tracks by name\n")
        print(f"{'Size':>6} | {'Python':>8} | {'Heapsort':>8} | {'Mergesort':>8}")
        print("-" * 60)

        for size in sizes:
            # Get subset of tracks
            subset = track_list[:size]
            key_func = lambda t: t['name']

            # Time each algorithm
            time_python, _ = time_sort(sorted, subset, key=key_func)
            time_heap, _ = time_sort(heap sort, subset, key=key_func)
            time_merge, _ = time_sort(merge sort, subset, key=key_func)
            time_quick, _ = time_sort(quick sort, subset, key=key_func)

            print(f"{'size':6d} | {time_python*1000:7.2f}ms | {time_heap*1000:7.2f}ms | {time_merge*1000:7.2f}ms | {time_quick*1000:7.2f}ms")

        print("\nNote: Python's sorted() uses Timsort (optimized C implementation)")
```

Benchmark on different data patterns

```
In [ ]: # Test on different input patterns
n = 1000
numbers = list(range(n))

patterns = [
    ("Random", random.sample(numbers, n)),
    ("Sorted", numbers.copy()),
    ("Reversed", numbers[::-1]),
    ("Nearly sorted", numbers[:980] + random.sample(numbers[980:], 20))
]

print("Benchmark: Different input patterns (n=1000)\n")
print(f"{'Pattern':>15} | {'Python':>8} | {'Heapsort':>8} | {'Mergesort':>8} | {'Quicksort':>8}")
print("-" * 70)

for pattern_name, data in patterns:
    time_python, _ = time_sort(sorted, data)
    time_heap, _ = time_sort(heapq, data)
    time_merge, _ = time_sort(mergesort, data)
    time_quick, _ = time_sort(quickselect, data)

    print(f"{'pattern_name':>15} | {time_python*1000:7.2f}ms | {time_heap*1000:7.2f}ms | {time_merge*1000:7.2f}ms | {time_quick*1000:7.2f}ms")

print("\nObservations:")
print("- Python (Timsort) is fastest on sorted/nearly-sorted (O(n) best)
```



```
print("- Quicksort can be slow on sorted data (bad pivot selection)")  
print("- Mergesort is consistent (always  $O(n \log n)$ )")  
print("- Heapsort is steady but not fastest")
```

Algorithm comparison table

Algorithm	Best	Average	Worst	Space	Stable	Adaptive	In-Place
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓	✓
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗	✗	✓
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	✗	✗
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗	Partial	✓
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	✓	✗

Key takeaways:

- **Heapsort:** Guaranteed $O(n \log n)$, minimal space, but not stable
- **Mergesort:** Guaranteed $O(n \log n)$, stable, but needs $O(n)$ space
- **Quicksort:** Fast average case, in-place, but $O(n^2)$ worst case
- **Timsort:** Best of all worlds for real-world data

When to use which algorithm

Use Python's `sorted()` or `.sort()` (Timsort)

- **Default choice for almost everything**
- Stable, fast, adaptive, well-tested
- Only reason not to: implementing your own for learning

Use Heapsort when:

- Need guaranteed $O(n \log n)$ worst case
- Minimal memory available ($O(1)$ extra)
- Don't need stability
- Example: Embedded systems, real-time constraints

Use Mergesort when:

- Need stability (equal elements maintain order)
- Need guaranteed $O(n \log n)$
- Have extra memory available
- Example: Sorting database records by multiple keys

Use Quicksort when:

- Average case performance matters more than worst case
- Limited memory (in-place sorting)
- Can use good pivot selection (random or median-of-three)
- Example: Systems programming, sorting large files

Special-purpose sorts

When comparison-based sorts aren't enough:

Counting Sort

- **When:** Sorting integers in small range $[0, k]$
- **Time:** $O(n + k)$
- **Space:** $O(k)$
- **Stable:** Yes
- **Example:** Sorting ages (0-120), grades (0-100)

Radix Sort

- **When:** Sorting integers or strings digit-by-digit
- **Time:** $O(nk)$ where k is number of digits
- **Space:** $O(n + k)$
- **Stable:** Yes
- **Example:** Sorting phone numbers, zip codes

Bucket Sort

- **When:** Data uniformly distributed over range
- **Time:** $O(n)$ average, $O(n^2)$ worst
- **Space:** $O(n)$
- **Example:** Sorting floating-point numbers in $[0, 1)$

Exercise 2: Benchmark and analyze

Task: Run benchmarks and answer analysis questions.

Requirements:

1. Create a list of 500 tracks sorted by artist name
2. Time how long it takes to sort this already-sorted list with:
 - Python's `sorted()`
 - Your `quicksort()` implementation
3. Create a reversed list (reverse the sorted list)
4. Time both algorithms on the reversed list
5. Answer in comments:
 - Which is faster on sorted data? Why?
 - Which is faster on reversed data? Why?
 - What does this tell you about Timsort vs Quicksort?

Commit required: Commit your benchmarks and analysis before class ends.

```
In [ ]: # YOUR CODE HERE
        # Run benchmarks and write analysis in comments

        # Analysis:
        # 1. Which is faster on sorted data?
```

Answer:

2. Which is faster on reversed data?

Answer:

3. What does this tell you about Timsort vs Quicksort?

Answer:

Complexity summary

Comparison-based sorts:

- **Lower bound:** $\Omega(n \log n)$ comparisons required
- **Heapsort, Mergesort, Timsort:** Achieve $O(n \log n)$ worst case
- **Quicksort:** $O(n \log n)$ average, $O(n^2)$ worst

Space complexity:

- **Heapsort, Quicksort:** $O(1)$ or $O(\log n)$ extra space
- **Mergesort, Timsort:** $O(n)$ extra space

Stability matters when:

- Sorting by multiple keys (sort by secondary, then primary)
- Preserving database record order
- UI consistency (same elements don't jump around)

Python's choice:

- Timsort combines best properties: stable, $O(n \log n)$ worst case, $O(n)$ best case on sorted data
- Reason Python's `sorted()` is so good!

Wrap-up: Sorting algorithm tradeoffs

You've learned:

-  Heapsort: $O(n \log n)$ guaranteed, $O(1)$ space, not stable
-  Mergesort: $O(n \log n)$ guaranteed, stable, $O(n)$ space
-  Quicksort: $O(n \log n)$ average, $O(n^2)$ worst, in-place
-  Timsort: Python's hybrid, best real-world performance
-  Empirical benchmarks show Timsort wins on typical data
-  Pivot selection matters for Quicksort (median-of-three best)
-  Algorithm selection depends on: guarantees, space, stability needs

Practical takeaway: Use Python's `sorted()` unless you have a specific reason not to.

Next: Exam 2 prep!

Don't forget: Commit both exercises before you leave!

Complexity checkpoints

Question 1: Why does Quicksort have $O(n^2)$ worst case?

- A) It always picks the wrong pivot
- B) Bad pivot selection creates unbalanced partitions
- C) It uses too much memory
- D) Python's implementation is slow

Question 2: Which sorting algorithm is stable?

- A) Heapsort
- B) Quicksort (standard version)
- C) Mergesort
- D) All of the above

Question 3: When does Timsort achieve $O(n)$ time?

- A) Never — it's always $O(n \log n)$
- B) On already-sorted or nearly-sorted data
- C) On reversed data
- D) On random data

Answers: B (unbalanced partitions), C (mergesort), B (sorted/nearly-sorted)

